

Objetivo

Preparar y validar un clúster Minikube de tres nodos para realizar operaciones de mantenimiento. Ejecutar el flujo cordon, drain y uncordon sobre un worker node y observar el efecto sobre los workloads. Verificar la reprogramación automática de Pods administrados por Deployments después del drenado de un nodo. Crear y analizar un PodDisruptionBudget para proteger la disponibilidad de una aplicación durante una interrupción voluntaria. Diagnosticar un drain bloqueado por un PodDisruptionBudget y restaurar correctamente el nodo al servicio.

Requisitos previos

- Haber completado la Práctica 2 y la Práctica complementaria 2.1.
- Tener Docker Desktop instalado y en ejecución.
- Tener Minikube y kubectl instalados y disponibles desde Git Bash.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Comprender Deployments, ReplicaSets, Pods, labels y selectors.
- Disponer de recursos suficientes para ejecutar un clúster Minikube de tres nodos.

Duración estimada

60 minutos

Introducción

En esta práctica simularás una ventana de mantenimiento de infraestructura en Kubernetes. Prepararás un clúster Minikube con un control plane y dos workers, desplegarás workloads de prueba y ejecutarás el ciclo cordon → drain → uncordon sobre uno de los nodos. Observarás cómo Kubernetes reprograma Pods administrados por Deployments y utilizarás un PodDisruptionBudget para comprobar cómo se protege la disponibilidad durante una interrupción voluntaria. Finalmente resolverás un escenario donde el drain queda bloqueado por una política demasiado restrictiva.

Instrucciones



Preparación del directorio y del clúster

En esta práctica crearás el directorio `lab3` y prepararás un clúster Minikube con tres nodos. La topología multinodo es necesaria para observar cómo Kubernetes

reprograma workloads cuando un worker queda fuera de servicio.

Crear el subdirectorio de la práctica

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor está activo, porque Minikube utilizará Docker para ejecutar los tres nodos del clúster.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el subdirectorio destinado a esta práctica.

```
mkdir -p /c/LABS/kubernetes/lab3
```

 Copiar

- **Paso 3.** Cambia la ubicación activa de Git Bash al nuevo directorio para mantener separados los manifiestos y evidencias del laboratorio.

```
cd /c/LABS/kubernetes/lab3
```

 Copiar

- **Paso 4.** Ejecuta `pwd` y confirma que la ruta activa corresponde exactamente al directorio de la práctica antes de continuar.

```
pwd
```

 Copiar

Salida esperada:

```
/c/LABS/kubernetes/lab3
```

 Copiar

Preparar el clúster de tres nodos

- **Paso 5.** Ejecuta el comando siguiente para conocer la topología actual del clúster y determinar si ya dispones de tres nodos activos.

```
kubectl get nodes
```

 Copiar

- **Paso 6.** Si el clúster actual tiene un solo nodo, elimina únicamente el perfil Minikube local para recrearlo con la topología requerida por este laboratorio.

```
minikube delete
```

 Copiar

IMPORTANTE: Este paso elimina los recursos de las prácticas anteriores almacenados dentro de Minikube. Los archivos YAML permanecen en `C:\LABS\kubernetes` y podrán reutilizarse si fueran necesarios.

- **Paso 7.** Ejecuta el comando siguiente para crear un clúster Minikube con un control plane y dos nodos adicionales utilizando Docker como driver.

```
minikube start \  
  --nodes=3 \  
  --driver=docker \  
  --cpus=2 \  
  --memory=2048
```

 Copiar

- **Paso 8.** Comprueba que los tres nodos aparecen en estado `Ready` antes de desplegar workloads para el ejercicio de mantenimiento.

```
kubectl get nodes -o wide
```

 Copiar

Salida esperada aproximada:

NAME	STATUS	ROLES	AGE	
VERSION				
minikube	Ready	control-plane	...	...
minikube-m02	Ready	<none>	...	...
minikube-m03	Ready	<none>	...	...

 Copiar

IMPORTANTE: No continúes si alguno de los nodos aparece como `NotReady`. El comportamiento de cordon, drain y reprogramación requiere que los tres nodos estén disponibles al inicio.



Tarea 1. Preparar workloads y establecer una línea base

En esta tarea desplegarás dos aplicaciones de prueba dentro de un namespace dedicado. Después observarás en qué nodos fueron programados los Pods para comparar su distribución antes y después del mantenimiento.

Tarea 1.1. Crear el namespace y los manifiestos

- **Paso 9.** Ejecuta el comando siguiente para crear el namespace `lab3`, aislando los workloads de esta práctica respecto de los recursos del sistema.

```
kubectl create namespace lab3
```

[Copiar](#)

- **Paso 10.** Crea el archivo donde almacenarás los dos Deployments utilizados durante las operaciones de mantenimiento.

```
touch deployments.yaml
```

[Copiar](#)

- **Paso 11.** Abre `deployments.yaml` en VS Code y agrega los dos Deployments siguientes con suficientes réplicas para observar la reprogramación entre workers.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-web
  namespace: lab3
  labels:
    app: app-web
spec:
  replicas: 4
  selector:
    matchLabels:
      app: app-web
  template:
    metadata:
      labels:
        app: app-web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "100m"
              memory: "128Mi"
```

 Copiar

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-backend
  namespace: lab3
  labels:
    app: app-backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: app-backend
  template:
    metadata:
```

```
labels:
  app: app-backend
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "50m"
          memory: "64Mi"
        limits:
          cpu: "100m"
          memory: "128Mi"
```

Tarea 1.2. Desplegar y observar la distribución

- **Paso 12.** Valida el archivo antes de aplicarlo para confirmar que ambos Deployments tienen una estructura aceptada por kubectl.

```
kubectl apply --dry-run=client -f
deployments.yaml
```

[Copiar](#)

- **Paso 13.** Aplica los manifiestos para crear los dos workloads dentro del namespace utilizado por esta práctica.

```
kubectl apply -f deployments.yaml
```

[Copiar](#)

- **Paso 14.** Espera a que los Deployments indiquen que todas sus réplicas se encuentran disponibles antes de registrar la distribución inicial.

```
kubectl rollout status deployment/app-web -n
lab3
kubectl rollout status deployment/app-backend -n
lab3
```

[Copiar](#)

- **Paso 15.** Ejecuta la consulta siguiente para identificar exactamente en qué nodo está ejecutándose cada Pod.

```
kubectl get pods -n lab3 -o wide
```

 Copiar

Resultado esperado:

Debes observar siete Pods en estado `Running`, distribuidos entre los nodos disponibles del clúster.

NOTA: El scheduler no garantiza una distribución perfectamente equilibrada. Lo importante es registrar qué Pods se encuentran en `minikube-m02`, porque ese será el nodo sometido a mantenimiento.

Tarea finalizada

Resultado esperado: Workloads desplegados y distribuidos correctamente en un clúster Minikube de tres nodos, estableciendo una línea base operativa.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 1 con IA](#)

🚧 Tarea 2. Ejecutar cordon y comprobar el efecto sobre scheduling

En esta tarea marcarás `minikube-m02` como no programable. Los Pods existentes continuarán ejecutándose, pero el scheduler dejará de colocar nuevos Pods sobre ese nodo.

Tarea 2.1. Marcar el nodo como SchedulingDisabled

- **Paso 16.** Ejecuta `kubectl cordon` sobre `minikube-m02` para impedir que el scheduler asigne nuevos Pods durante la ventana de mantenimiento.

```
kubectl cordon minikube-m02
```

 Copiar

- **Paso 17.** Comprueba inmediatamente el estado de los nodos y localiza la indicación `SchedulingDisabled` junto al worker seleccionado.

```
kubectl get nodes
```

 Copiar

Salida esperada aproximada:

NAME	STATUS	ROLES
minikube	Ready	control-plane ...
minikube-m02	Ready,SchedulingDisabled	<none>
...		
minikube-m03	Ready	<none>
...		

 Copiar

- **Paso 18.** Ejecuta el comando siguiente para comprobar que los Pods que ya estaban en `minikube-m02` continúan ejecutándose después del cordon.

```
kubectl get pods -n lab3 -o wide | grep  
minikube-m02
```

 Copiar

Tarea 2.2. Confirmar que los nuevos Pods evitan el nodo

- **Paso 19.** Escala temporalmente `app-web` de cuatro a seis réplicas para forzar al scheduler a crear dos Pods nuevos mientras `minikube-m02` está cordonado.

```
kubectl scale deployment app-web \  
-n lab3 \  
--replicas=6
```

 Copiar

- **Paso 20.** Observa la ubicación de los seis Pods y verifica que las nuevas instancias no hayan sido programadas en `minikube-m02`.

```
kubectl get pods -n lab3 -l app=app-web -o wide
```

 Copiar

- **Paso 21.** Regresa `app-web` a cuatro réplicas para continuar con el escenario de mantenimiento definido para el resto del laboratorio.

```
kubectl scale deployment app-web \
  -n lab3 \
  --replicas=4
```

 Copiar

Concepto clave: `cordon` no expulsa ni reinicia Pods existentes. Únicamente evita nuevas asignaciones sobre el nodo hasta que vuelva a ser habilitado.

Tarea finalizada

Resultado esperado: Nodo marcado con `cordon`, confirmando que los Pods existentes continúan activos y los nuevos evitan el nodo en mantenimiento.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 2 con IA](#)

Tarea 3. Crear un PodDisruptionBudget y preparar el escenario de bloqueo

En esta tarea crearás dos PodDisruptionBudgets. Uno permitirá una interrupción controlada y el segundo bloqueará cualquier evicción para generar un escenario real de troubleshooting.

Tarea 3.1. Crear el PDB de app-web

- **Paso 22.** Crea el archivo `pdb.yaml`, donde definirás las políticas de disponibilidad utilizadas durante el drain.

```
touch pdb.yaml
```

 Copiar

- **Paso 23.** Abre `pdb.yaml` en VS Code y agrega un presupuesto para `app-web` que conserve al menos tres de sus cuatro réplicas disponibles.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pdb-app-web
  namespace: lab3
spec:
  minAvailable: 3
  selector:
    matchLabels:
      app: app-web
```

 Copiar

- **Paso 24.** Aplica el manifiesto y consulta el presupuesto para comprobar cuántas interrupciones voluntarias están permitidas actualmente.

```
kubectl apply -f pdb.yaml
kubectl get pdb -n lab3
```

 Copiar

Salida esperada aproximada:

NAME	MIN AVAILABLE	MAX UNAVAILABLE
ALLOWED DISRUPTIONS		
pdb-app-web	3	N/A

 Copiar

Tarea 3.2. Crear un PDB restrictivo para app-backend

- **Paso 25.** Agrega al final de `pdb.yaml` el recurso siguiente, que exige mantener disponibles las tres réplicas actuales de `app-backend`.

```
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pdb-app-backend-strict
  namespace: lab3
spec:
  minAvailable: 3
  selector:
    matchLabels:
      app: app-backend
```

 Copiar

- **Paso 26.** Aplica nuevamente el archivo y confirma que el segundo PDB no permite ninguna disrupción mientras el Deployment tenga solamente tres réplicas.

```
kubectl apply -f pdb.yaml
kubectl get pdb -n lab3
```

 Copiar

Salida esperada aproximada:

NAME	MIN AVAILABLE	ALLOWED
DISRUPTIONS		
pdb-app-web	3	1
pdb-app-backend-strict	3	0

 Copiar

IMPORTANTE: `ALLOWED DISRUPTIONS: 0` significa que una evicción voluntaria de cualquier Pod protegido por ese PDB será rechazada mientras no aumente la disponibilidad.

Tarea finalizada

Resultado esperado: PodDisruptionBudgets configurados correctamente, identificando cuándo una política permite o bloquea disrupciones voluntarias.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 3 con IA](#)

Tarea 4. Ejecutar drain, diagnosticar el bloqueo y reprogramar workloads

En esta tarea intentarás vaciar `minikube-m02`. El primer intento quedará bloqueado por el PDB restrictivo de `app-backend`; después corregirás la condición y completarás el mantenimiento.

Tarea 4.1. Intentar el drain y analizar el error

- **Paso 27.** Ejecuta el drain con un timeout controlado para evitar que el comando permanezca reintentando indefinidamente cuando encuentre el PDB restrictivo.

```
kubectl drain minikube-m02 \  
  --ignore-daemonsets \  
  --delete-emptydir-data \  
  --timeout=60s
```

 Copiar

Salida esperada aproximada:

```
error when evicting pods/...:  
Cannot evict pod as it would violate the pod's  
disruption budget.
```

 Copiar

- **Paso 28.** Ejecuta la consulta siguiente para identificar qué PDB tiene cero disrupciones permitidas y relacionarlo con el workload que bloqueó el drain.

```
kubectl get pdb -n lab3
```

 Copiar

- **Paso 29.** Comprueba la cantidad de réplicas disponibles de `app-backend` para relacionar el valor `minAvailable: 3` con sus tres Pods actuales.

```
kubectl get deployment app-backend -n lab3
```

 Copiar

Tarea 4.2. Resolver correctamente la restricción

- **Paso 30.** Escala `app-backend` a cuatro réplicas para disponer de una instancia adicional y permitir que el PDB autorice una interrupción durante el mantenimiento.

```
kubectl scale deployment app-backend \  
-n lab3 \  
--replicas=4
```

 Copiar

- **Paso 31.** Espera hasta que el Deployment confirme las cuatro réplicas disponibles antes de volver a intentar el drenado.

```
kubectl rollout status deployment/app-backend -n  
lab3
```

 Copiar

- **Paso 32.** Comprueba nuevamente el PDB y confirma que ahora existe al menos una interrupción permitida.

```
kubectl get pdb -n lab3
```

 Copiar

Tarea 4.3. Completar el drain

- **Paso 33.** Ejecuta nuevamente el comando de drain para evacuar los Pods administrados que todavía permanecen en `minikube-m02`.

```
kubectl drain minikube-m02 \  
--ignore-daemonsets \  
--delete-emptydir-data \  
--timeout=120s
```

 Copiar

Salida esperada final:

```
node/minikube-m02 drained
```

 Copiar

- **Paso 34.** Ejecuta la consulta siguiente para comprobar que los Pods del namespace `lab3` fueron reprogramados en nodos disponibles y ya no permanecen en el worker drenado.

```
kubectl get pods -n lab3 -o wide
```

 Copiar

- **Paso 35.** Ejecuta el filtro siguiente para confirmar que ningún Pod de usuario del laboratorio continúa ejecutándose en `minikube-m02`.

```
kubectl get pods -n lab3 -o wide | grep  
minikube-m02
```

 Copiar

Resultado esperado:

La última consulta no debe devolver Pods del namespace `lab3`.

NOTA: Los Pods administrados por DaemonSets del sistema pueden continuar visibles en `minikube-m02`. El parámetro `--ignore-daemonsets` indica precisamente que no deben impedir el drenado.

Tarea finalizada

Resultado esperado: Drain completado después de diagnosticar y resolver el bloqueo del PDB, reprogramando los Pods en nodos disponibles.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 4 con IA](#)

✓ Tarea 5. Reincorporar el nodo y validar el estado final

En esta tarea ejecutarás `uncordon` para habilitar nuevamente el worker y comprobarás un comportamiento importante: Kubernetes no redistribuye automáticamente los Pods existentes cuando un nodo vuelve a estar disponible.

Tarea 5.1. Ejecutar uncordon

- **Paso 36.** Ejecuta el comando siguiente para reincorporar `minikube-m02` al conjunto de nodos disponibles para nuevas asignaciones.

```
kubect1 uncordon minikube-m02
```

[Copiar](#)

- **Paso 37.** Comprueba que el nodo volvió al estado `Ready` y que ya no aparece la indicación `SchedulingDisabled`.

```
kubect1 get nodes
```

[Copiar](#)

Salida esperada aproximada:

NAME	STATUS	ROLES	AGE
minikube	Ready	control-plane	...
minikube-m02	Ready	<none>	...
minikube-m03	Ready	<none>	...

[Copiar](#)

Tarea 5.2. Observar el comportamiento después del uncordon

- **Paso 38.** Ejecuta la consulta siguiente inmediatamente después del `uncordon` y observa que los Pods existentes permanecen donde fueron reprogramados durante el drain.

```
kubect1 get pods -n lab3 -o wide
```

[Copiar](#)

Concepto clave: `uncordon` permite que el nodo vuelva a recibir Pods nuevos, pero no provoca un rebalanceo automático de los Pods que ya están ejecutándose correctamente en otros nodos.

- **Paso 39.** Ejecuta un rolling restart sobre ambos Deployments para provocar la recreación progresiva de Pods y permitir que el scheduler vuelva a considerar todos los nodos disponibles.

```
kubectl rollout restart deployment/app-web -n lab3  
kubectl rollout restart deployment/app-backend -n lab3
```

 Copiar

- **Paso 40.** Espera hasta que ambos rollouts terminen correctamente antes de revisar nuevamente la distribución de los Pods.

```
kubectl rollout status deployment/app-web -n lab3  
kubectl rollout status deployment/app-backend -n lab3
```

 Copiar

- **Paso 41.** Ejecuta la consulta siguiente para observar la nueva ubicación de los Pods después de su recreación.

```
kubectl get pods -n lab3 -o wide
```

 Copiar

Tarea 5.3. Ejecutar la verificación final

- **Paso 42.** Copia y ejecuta el bloque completo para confirmar automáticamente el estado de los nodos, Deployments, Pods y PDBs después del ciclo de mantenimiento.

```
echo "=== Verificacion final de la Practica 3
=== "

NOT_READY=$(kubectl get nodes --no-headers \
  | awk '$2 !~ /^Ready/ {count++} END {print
count+0}')

if [ "$NOT_READY" = "0" ]; then
  echo "✅ Todos los nodos estan disponibles"
else
  echo "❌ Existe al menos un nodo no
disponible"
fi

if kubectl get nodes --no-headers | grep -q
"SchedulingDisabled"; then
  echo "❌ Existe un nodo con
SchedulingDisabled"
else
  echo "✅ Todos los nodos aceptan scheduling"
fi

WEB_READY=$(kubectl get deployment app-web -n
lab3 \
  -o jsonpath='{.status.readyReplicas}')

WEB_DESIRED=$(kubectl get deployment app-web -n
lab3 \
  -o jsonpath='{.spec.replicas}')

if [ "$WEB_READY" = "$WEB_DESIRED" ]; then
  echo "✅ app-web disponible:
$WEB_READY/$WEB_DESIRED"
else
  echo "❌ app-web incompleto:
$WEB_READY/$WEB_DESIRED"
fi

BACK_READY=$(kubectl get deployment app-backend
-n lab3 \
  -o jsonpath='{.status.readyReplicas}')

BACK_DESIRED=$(kubectl get deployment app-
backend -n lab3 \
```

 Copiar

```
-o jsonpath='{.spec.replicas}')
```

```
if [ "$BACK_READY" = "$BACK_DESIRED" ]; then
    echo "✅ app-backend disponible:
$BACK_READY/$BACK_DESIRED"
else
    echo "❌ app-backend incompleto:
$BACK_READY/$BACK_DESIRED"
fi

echo ""
echo "Distribucion final de Pods:"
kubectl get pods -n lab3 -o wide

echo ""
echo "PodDisruptionBudgets:"
kubectl get pdb -n lab3

echo "=== Fin de verificacion ==="
```

Salida esperada aproximada:

```
=== Verificacion final de la Practica 3 ===
✅ Todos los nodos estan disponibles
✅ Todos los nodos aceptan scheduling
✅ app-web disponible: 4/4
✅ app-backend disponible: 4/4
...
=== Fin de verificacion ===
```

[Copiar](#)

IMPORTANTE: Conserva el clúster Minikube de tres nodos. Esta topología será útil para las siguientes prácticas relacionadas con namespaces, scheduling y troubleshooting.

Tarea finalizada

Resultado esperado: Nodo reincorporado mediante `uncordon`, workloads disponibles y clúster validado después del ciclo completo de mantenimiento.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 5 con IA](#)

Resolución de problemas

Problema 1. El clúster no inicia con tres nodos

Síntoma: `minikube start --nodes=3` falla, uno de los nodos no aparece o permanece en estado `NotReady`.

Causa probable: Docker Desktop no dispone de CPU o memoria suficientes para ejecutar simultáneamente los tres contenedores utilizados como nodos.

Solución:

Comprueba primero el estado de Docker y del perfil Minikube.

```
docker info
minikube status
kubectl get nodes
```

 Copiar

Si el host tiene recursos limitados, cierra aplicaciones no necesarias y revisa la asignación de recursos disponible para Docker Desktop antes de recrear el clúster.

Problema 2. El drain permanece bloqueado

Síntoma: `kubectl drain` repite mensajes de evicción y termina por timeout indicando que un Pod no puede ser eliminado.

Causa probable: Un PodDisruptionBudget tiene `ALLOWED DISRUPTIONS: 0`, por lo que Kubernetes protege todas las réplicas disponibles del workload.

Solución:

Identifica el PDB bloqueante y compara sus requisitos con las réplicas actuales.

```
kubectl get pdb -n lab3
kubectl get deployments -n lab3
```

 Copiar

Si `podb-app-backend-strict` muestra cero interrupciones permitidas, escala `app-backend` antes de reintentar:

```
kubectl scale deployment app-backend \
  -n lab3 \
  --replicas=4

kubectl rollout status deployment/app-backend -n lab3
kubectl get podb -n lab3
```

[Copiar](#)

Problema 3. Los Pods permanecen en Pending después del drain

Síntoma: Uno o más Pods reprogramados permanecen en `Pending` después de evacuar `minikube-m02`.

Causa probable: Los nodos restantes no tienen capacidad suficiente para satisfacer los requests de CPU o memoria de todos los Pods.

Solución:

Identifica el Pod afectado y revisa los eventos de scheduling.

```
kubectl get pods -n lab3
POD_NAME=$(kubectl get pods -n lab3 \
  --field-selector=status.phase=Pending \
  -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod "$POD_NAME" -n lab3
```

[Copiar](#)

Revisa la sección `Events` para determinar qué recurso impide la programación.

Problema 4. Después del uncordon los Pods no regresan al nodo

Síntoma: `minikube-m02` aparece nuevamente como `Ready`, pero todos los Pods continúan ejecutándose en los otros nodos.

Causa probable: Este es el comportamiento normal de Kubernetes. El scheduler no mueve Pods sanos solamente para equilibrar nuevamente la carga.

Solución:

Confirma primero que el nodo acepta scheduling:

```
kubectl get nodes
```

 Copiar

Si necesitas provocar una nueva decisión del scheduler, recrea progresivamente los Pods mediante un rolling restart:

```
kubectl rollout restart deployment/app-web -n lab3  
kubectl rollout restart deployment/app-backend -n  
lab3
```

 Copiar

Problema 5. El nodo continúa en SchedulingDisabled

Síntoma: Después de finalizar el mantenimiento, `kubectl get nodes` continúa mostrando `Ready,SchedulingDisabled` para `minikube-m02`.

Causa probable: El comando `uncordon` no se ejecutó o terminó con un error.

Solución:

Ejecuta nuevamente la reincorporación y verifica inmediatamente el estado:

```
kubectl uncordon minikube-m02  
kubectl get nodes
```

 Copiar

El nodo debe mostrarse como `Ready` sin la indicación `SchedulingDisabled`.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica tendrás un clúster Minikube de tres nodos operativo, habrás realizado el mantenimiento controlado de un worker mediante `cordons`, `drain` y `uncordon`, observado la reprogramación automática de workloads y diagnosticado un bloqueo de `drain` provocado por un `PodDisruptionBudget`.

Notas y consideraciones

- Esta práctica requiere tres nodos Minikube porque el objetivo principal es observar la reprogramación de Pods entre workers durante una operación real de mantenimiento.

- El clúster creado en esta práctica debe conservarse para laboratorios posteriores relacionados con nodos, scheduling y troubleshooting.
- El PodDisruptionBudget se utilizará únicamente para demostrar el comportamiento de las interrupciones voluntarias; no reemplaza una estrategia completa de alta disponibilidad.
- No uses kubectl drain con `–force` durante las actividades guiadas, porque el objetivo es respetar y analizar las protecciones definidas por Kubernetes.

URL de referencia

- [Safely Drain a Node](#)
- [Disruptions](#)
- [PodDisruptionBudget](#)
- [kubectl drain](#)